

VERUNUM

TAB5 / R503 Hardware Developer Integration Guide

Protocol: REST over HTTP/HTTPS only

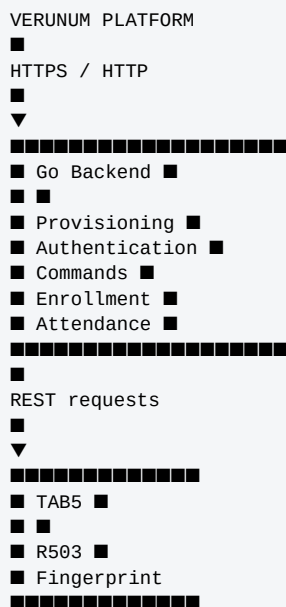
Audience: Hardware / firmware developer

Purpose: Implement the TAB5 fingerprint terminal so that it communicates with the Verunum backend exactly as the platform expects.

Important: This guide describes the current REST-only hardware architecture. The previous WebSocket hardware contract is superseded for this version. Do not implement a WebSocket device protocol for this build.

1. System Goal

The TAB5 is the physical attendance terminal. It uses the R503 fingerprint sensor locally, communicates with Verunum using REST requests, receives commands by polling, reports enrollment results, submits attendance events, and queues attendance locally when the network is unavailable.



2. Base URL

Production:

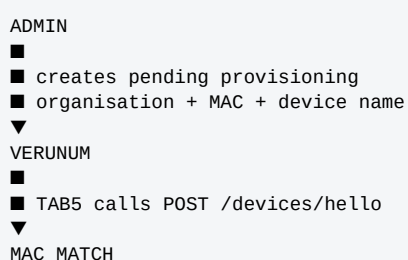
https://YOUR-HOST/api/v1

Initial EC2 testing:

http://YOUR_EC2_PUBLIC_IP:8080/api/v1

The hardware should keep the API base URL configurable rather than hard-coded into application logic. HTTPS should be used when the backend is exposed as a production service.

3. Device Lifecycle



```
■
■■■ create permanent device
■■■ bind device to organisation
■■■ consume pending provisioning
■■■ generate API key
```

```
▼
TAB5 receives API key ONCE
```

```
■
■■■ securely stores API key
```

```
▼
authenticated REST requests
```

Critical provisioning rule: the API key is returned only during successful provisioning. It must not be hard-coded into firmware source code.

4. API Authentication

After provisioning, every device-authenticated request must include the API key in the HTTP header:

```
X-Device-Key: <api-key>
```

The backend stores a hash of the device API key and validates the supplied key. A revoked device must be rejected.

5. First-Time Device Provisioning

Step 1: An organisation administrator creates a pending device provisioning record in the Verunum dashboard. The record contains the organisation, device MAC address, device name and optional location.

Step 2: The TAB5 sends its identity to the backend.

```
POST /api/v1/devices/hello
Content-Type: application/json
```

```
{
  "mac_address": "AA:BB:CC:DD:EE:FF",
  "device_name": "Main Gate TAB5",
  "protocol_version": 1,
  "firmware_version": "1.0.0"
}
```

Step 3: The backend compares the MAC address against pending provisioning records.

Step 4: If matched, the backend creates the permanent device, binds it to the organisation, consumes the pending record and generates the device API key.

Step 5: The API key is returned once.

```
HTTP 201 Created
```

```
{
  "status": "provisioned",
  "device_id": "dev_123",
  "organization_id": 1,
  "device_name": "Main Gate TAB5",
  "mac_address": "AA:BB:CC:DD:EE:FF",
  "api_key": "<one-time-secret>",
  "protocol_version": 1,
  "firmware_version": "1.0.0",
  "message": "Store this API key securely. It is returned only during provisioning."
}
```

Unknown MAC: HTTP 404.

Already-provisioned MAC: HTTP 409.

6. Heartbeat

The TAB5 periodically tells the backend that it is alive.

```
POST /api/v1/devices/{device_id}/heartbeat
X-Device-Key: <api-key>
```

Response: HTTP 204 No Content

Recommended firmware interval: every 30 seconds. The dashboard considers a device online when its last heartbeat is no older than approximately 90 seconds.

7. Command Polling

Because REST is request/response based, the backend does not push commands directly to the TAB5. Instead, the TAB5 polls for pending commands.

```
GET /api/v1/devices/{device_id}/commands
X-Device-Key: <api-key>
```

Example response:

```
{
  "commands": [
    {
      "id": 41,
      "command_type": "enroll.start",
      "payload": {
        "request_id": "req-123",
        "user_id": "user-uuid",
        "finger_index": 1,
        "timeout_seconds": 30
      },
      "created_at": "2026-09-25T00:00:00Z"
    }
  ]
}
```

Recommended command polling interval: every 2–5 seconds during normal operation, with backoff when the network is unavailable.

Pending commands remain on the backend until the device acknowledges them. This is what allows an offline device to receive a command after it comes back online.

8. Command Acknowledgement

```
POST /api/v1/devices/{device_id}/commands/{command_id}/ack
X-Device-Key: <api-key>
Content-Type: application/json
```

```
{
  "status": "acked"
}
```

A command should only be acknowledged after the TAB5 has accepted or completed the requested operation according to the command semantics.

9. Fingerprint Enrollment

Dashboard: the administrator chooses a person and a device and starts fingerprint enrollment.

The backend creates an enrollment command for that device.

```
POST /api/v1/devices/{device_id}/enroll-request
Content-Type: application/json
```

```
{
  "user_id": "user-uuid",
  "finger_index": 1,
  "timeout_seconds": 30
}
```

The TAB5 discovers the command through command polling:

```
GET /api/v1/devices/{device_id}/commands
```

When the TAB5 receives enroll.start, it uses the R503 to perform the fingerprint operation locally.

The TAB5 then reports the result:

```
POST /api/v1/devices/{device_id}/enrollment/result
X-Device-Key: <api-key>
Content-Type: application/json

{
  "command_id": 41,
  "request_id": "req-123",
  "user_id": "user-uuid",
  "finger_index": 1,
  "success": true
}
```

Biometric rule: raw fingerprint templates are not sent to the Verunum backend under this contract. The biometric operation remains local to the TAB5/R503.

10. Attendance

The TAB5 performs fingerprint matching locally. When a fingerprint is matched to a Verunum user, the terminal sends an attendance event to the backend.

```
POST /api/v1/devices/{device_id}/attendance
X-Device-Key: <api-key>
Content-Type: application/json

{
  "user_id": 12,
  "event_id": "evt-unique-001",
  "event": "clock_in",
  "timestamp": "2026-09-25T08:15:00Z",
  "method": "fingerprint"
}
```

Example response:

HTTP 201 Created

```
{
  "id": 501,
  "attendance_status": "on_time"
}
```

event_id is mandatory for reliable synchronization. If the same event is retried, the original event_id must be preserved so the backend can identify duplicates.

11. Offline Attendance

```
NETWORK AVAILABLE
■
▼
Fingerprint match
■
▼
POST /attendance
■
▼
Backend stores event

NETWORK UNAVAILABLE
■
▼
Fingerprint match
■
▼
Store event locally
■
■■■ preserve event_id
```

■■■ preserve timestamp

■

▼

Network returns

■

▼

Retry POST /attendance

■

▼

Backend deduplicates by event_id

For multiple queued events, use the batch endpoint:

POST /api/v1/devices/{device_id}/attendance/batch

X-Device-Key: <api-key>

Content-Type: application/json

```
[
{
"user_id": 12,
"event_id": "evt-001",
"event": "clock_in",
"timestamp": "2026-09-25T08:15:00Z",
"method": "fingerprint"
}
]
```

12. Complete Hardware Communication Flow

```
POWER ON
▼
Load configuration
API base URL
stored device credential (if provisioned)
▼
Have API key?
NO POST /devices/hello
MAC matched?
NO YES
retry/ receive
report API key
store key
YES
▼
heartbeat every 30s
poll commands fingerprint scan
enroll.start local match
R503 attendance
enrollment/result POST attendance
command acknowledgement
repeat
```

13. API Endpoint Reference

Method	Endpoint	Purpose
POST	/api/v1/devices/hello	First-time MAC-based provisioning.
POST	/api/v1/devices/{device_id}/heartbeat	Device liveness heartbeat.
GET	/api/v1/devices/{device_id}/commands	Poll pending commands.
POST	/api/v1/devices/{device_id}/commands/{command_id}/ack	Acknowledge command.
POST	/api/v1/devices/{device_id}/enroll-request	Create fingerprint enrollment command.
POST	/api/v1/devices/{device_id}/enrollment/result	Return enrollment result.
POST	/api/v1/devices/{device_id}/attendance	Submit one attendance event.
POST	/api/v1/devices/{device_id}/attendance/batch	Synchronize queued attendance events.

14. HTTP Status Handling


```
■■■ network failure ■■■ OFFLINE_QUEUE
```

```
OFFLINE_QUEUE
```

```
■
```

```
■■■ keep event_id
```

```
■■■ keep timestamp
```

```
■■■ retry after network recovery
```

```
■
```

```
▼
```

```
READY
```

18. Hardware Developer Checklist

- Configure the Verunum API base URL.
- Read the device's MAC address reliably.
- Send POST /api/v1/devices/hello on first provisioning.
- Parse the provisioning response and securely persist device_id and api_key.
- Do not hard-code production credentials.
- Send X-Device-Key on authenticated requests.
- Implement heartbeat every 30 seconds.
- Implement command polling every 2-5 seconds with network-failure backoff.
- Implement enroll.start.
- Use R503 locally for fingerprint capture.
- Implement enrollment result reporting.
- Implement local fingerprint matching.
- Generate a unique attendance event_id.
- Preserve event_id when retrying.
- Implement offline attendance storage.
- Implement batch synchronization.
- Implement command acknowledgement.
- Implement HTTP status/error handling.
- Never transmit raw fingerprint templates.

19. End-to-End Hardware Acceptance Test

1. Admin creates an organisation.
2. Admin creates a pending device with the TAB5 MAC address.
3. TAB5 boots without an API key.
4. TAB5 POSTs /devices/hello.
5. Backend matches the MAC.
6. Backend creates and binds the device.
7. Backend returns API key once.
8. TAB5 securely stores the key.
9. TAB5 sends heartbeat.
10. TAB5 polls commands.
11. Admin starts enrollment.
12. TAB5 receives enroll.start.
13. R503 captures the fingerprint locally.
14. TAB5 POSTs enrollment/result.
15. User presents fingerprint.
16. TAB5 matches locally.
17. TAB5 POSTs attendance with event_id.
18. Backend stores the attendance.
19. Disconnect network.
20. User presents fingerprint again.
21. TAB5 stores attendance locally.
22. Restore network.

23. TAB5 retries the same event_id.
24. Backend stores it once.
25. TAB5 continues heartbeat and command polling.

20. Final Rule for the Firmware Developer

The TAB5 is a REST client. It does not maintain a WebSocket connection to Verunum in this version. All device communication is ordinary HTTP/HTTPS request/response communication.

The backend is authoritative for device identity, organisation binding, provisioning, commands and attendance persistence. The TAB5/R503 is responsible for local fingerprint capture/matching and temporary offline attendance storage.

If an endpoint or JSON field in the implementation differs from this guide, treat the actual deployed REST contract as the source of truth and update this document before firmware integration proceeds.